

Turtle Graphics Project, Spring 2026

Tong Yi

Contents

1	CopyRight Warning	1
2	Example of Separating Declaration, Definition, and Test Code: FuelTank Class	2
2.1	FuelTank.hpp	2
2.2	FuelTank.cpp	2
2.3	FuelTankTest.cpp	3
3	Overview of Turtle Graphics Project	4
4	Task A: Implement Floor Class	5
4.1	Download Floor.hpp	5
4.2	Implement Floor.cpp	6
4.3	Test Floor.cpp using FloorUnitTest.cpp	6
4.4	Submission for Task A	13
5	Task B: Implement Turtle Class	14
5.1	Download Turtle.hpp	14
5.2	Implement Turtle.cpp	18
5.3	Test Turtle.cpp using TurtleUnitTest.cpp	18
5.4	Submission for Task B	24
6	Task C: Implement TurtleGraphics Class	24
6.1	Download TurtleGraphics.hpp	24
6.2	Implement TurtleGraphics.cpp	25
6.3	Download input3.txt	25
6.4	Test TurtleGraphics.cpp	25
6.5	Submission for Task C	25

1 CopyRight Warning

1. This is copyrighted materials; you are not allowed to upload to the Internet.
2. Our project is different from similar products in Internet.
 - (a) Ask help only from teaching staff of this course.

- (b) Use solutions from artificial intelligence like ChatGPT or online tutoring websites like, but not limited to, chegg.com, violates academic integrity and is not allowed.

2 Example of Separating Declaration, Definition, and Test Code: FuelTank Class

For robust software engineering practices, especially in large projects, the standard C++ approach is to divide the code into three distinct files:

Header File (.h or .hpp): Contains the declaration of the class (the class blueprint, member variables, and function prototypes).

Source File (.cpp): Contains the definitions (implementation logic) of the class's member functions.

Test File (.cpp): Contains the main() function and the code dedicated to testing the functionality of the class.

This separation improves compile times (only changed implementation files need recompilation), enhances code organization, and facilitates code reuse across different parts of a project.

Code link: <https://onlinegdb.com/MzXcjYT2S->.

In online compilers like onlinegdb, the primary source file containing the `main` function must be named `main.cpp`. When working on your local computer, you have more flexibility and can name the test file something more descriptive, such as `FuelTankTest.cpp`. Naming the file `FuelTankTest.cpp` is generally preferred over `TestFuelTank.cpp` because it keeps related files grouped together when sorted alphabetically. This is a common practice in many C++ development environments to improve project organization and scannability.

2.1 FuelTank.hpp

```
1 #ifndef FUEL_TANK_H
2 #define FUEL_TANK_H
3 class FuelTank {
4 public:
5     FuelTank();
6     FuelTank(int startLevel);
7     int getGasLevel() const;
8     void fillGas(int numGallons);
9     void sendGas(int numGallons);
10
11 private:
12     int currGasLevel;
13 };
14 #endif
```

2.2 FuelTank.cpp

```
1 #include "FuelTank.hpp"
```

```

2
3 FuelTank::FuelTank() {
4     //int currGasLevel = 0; //wrong, with int before currGasLevel, the currGasLevel is
       a local variable
5     //for default constructor,
6     //that statement does not initialize data member currGasLevel
7     currGasLevel = 0;
8 }
9
10 FuelTank::FuelTank(int startGasLevel) {
11     if (startGasLevel >= 0) {
12         currGasLevel = startGasLevel;
13     }
14     else {
15         currGasLevel = 0;
16     }
17 }
18
19 int FuelTank::getGasLevel() const {
20     return currGasLevel;
21 }
22
23 void FuelTank::sendGas(int gallons) {
24     if (currGasLevel >= gallons) {
25         currGasLevel -= gallons;
26     }
27 }
28
29 void FuelTank::fillGas(int gallons) {
30     currGasLevel += gallons;
31 }

```

2.3 FuelTankTest.cpp

```

1 //Sample output:
2 //gas level of an empty fuel tank is 0
3 //add 5 gallons to tank
4 //send 3 gallons out from tank
5 //current gas level is 2
6
7 #include "FuelTank.hpp"
8 #include <iostream>
9
10 int main() {
11     FuelTank tank(0);

```

```

12
13     std::cout << "gas level of an empty fuel tank is "
14         << tank.getGasLevel() << std::endl;
15
16     std::cout << "add 5 gallons to tank" << std::endl;
17     tank.fillGas(5);
18
19     std::cout << "send 3 gallons out from tank" << std::endl;
20     tank.sendGas(3);
21
22     std::cout << "current gas level is ";
23     std::cout << tank.getGasLevel() << std::endl;
24
25     return 0;
26 }

```

To run the code,

```

1 g++ -o run FuelTank.cpp FuelTankTest.cpp
2 ./run

```

3 Overview of Turtle Graphics Project

This project involves three classes: Floor, Turtle, and TurtleGraphics.

- Floor is a square-shape grid with characters '*' and space character ' '. There are operations like clear, mark, at to operate on the grid.
- Turtle lets a turtle lift up or put down a pen, turning left or right, and move.
- TurtleGraphics lets a turtle to draw character '*' on the floor by turning, moving, and lifting up or putting down a pen properly.

To run the project, we do the following.

1. Build a directory in your computer by entering the following command with return key in a terminal.

```
mkdir TurtleGraphics
```

2. Move to the directory by pressing the following command with return key in a terminal.

```
cd TurtleGraphics
```

3. Download the required files and save them in the above directory.
4. You implement Floor, Turtle, and TurtleGraphics classes. Then test TurtleGraphics class. That is,
 - (a) Define Floor.cpp in Task A.

- (b) Define Turtle.cpp in Task B.
- (c) Define TurtleGraphics.cpp in Task C.
- (d) Define TurtleGraphicsTest.cpp in Task D.

4 Task A: Implement Floor Class

4.1 Download Floor.hpp

Download link: [Floor.hpp](#). Its contents are as follows.

```

1 #ifndef FLOOR_HPP
2 #define FLOOR_HPP
3
4 #include <vector>
5 #include <string>
6
7 /**
8  * @class Floor
9  * @brief Manages a 2D character grid with specific labeling and safety checks.
10  */
11 class Floor {
12 public:
13     /**
14      * @brief Default constructor.
15      * Initializes a default 20 x 20 grid filled with spaces (' ').
16      */
17     Floor();
18
19     /**
20      * @brief Parameterized constructor.
21      * @param size The desired dimension for a square grid.
22      * @note If size < 10, a default 20x20 grid is created instead.
23      */
24     Floor(int size);
25
26     /** @return The number of rows in the grid. */
27     int getNumRows() const;
28
29     /** @return The number of columns in the grid. */
30     int getNumCols() const;
31
32     /**
33      * @brief Places a character at the specified coordinates.
34      * @param row The row index from 0 to getNumRows() - 1.
35      * @param col The column index from 0 to getNumCols() - 1.
36      * @param ch The character to place.

```

```

37     * @note If indices are out of bounds, the operation is silently ignored.
38     */
39 void mark(int row, int col, char ch);
40
41 /**
42  * @brief Retrieves the character at the specified coordinates.
43  * @param row The row index from 0 to getNumRows() - 1.
44  * @param col The column index from 0 to getNumCols() - 1.
45  * @return The character at (row, col), or the null character '\0' if out of
46  *         bounds.
47  */
48 char at(int row, int col) const;
49
50 /**
51  * @brief Resets all cells in the grid to a space (' ').
52  */
53 void clear();
54
55 /**
56  * @brief Returns a string representation of the grid with coordinate labels.
57  *
58  * Labeling Rules:
59  * 1. Designed for grids up to 99x99.
60  * 2. The tens digit for row/column indices is displayed ONLY at multiples
61  *    of 10 (e.g., 10, 20, 30), appearing directly above or to the left.
62  * 3. The units digit is displayed for every row and column.
63  *
64  * @return A formatted string containing the labels and grid content.
65  */
66 std::string to_string() const;
67
68 private:
69     std::vector<std::vector<char>> grid;
70 };
71 #endif

```

4.2 Implement Floor.cpp

Read the requirement of Floor.hpp. Implement the constructors and methods in Floor.cpp.

4.3 Test Floor.cpp using FloorUnitTest.cpp

Download link: [FloorUnitTest.cpp](#) – its contents are as follows. The purpose is to test the constructors and methods of Floor.cpp.

```

1 //File name: FloorUnitTest.cpp
2 #include <iostream>
3 #include <vector>
4 #include <cstdlib> //srand
5 #include <ctime> //time
6 #include <sstream> //std::ostringstream
7 #include "Floor.hpp"
8 //g++ -o floor Floor.cpp FloorUnitTest.cpp
9 //test default constructor using
10 //./floor A
11
12 const int SIZE = 16;
13 Floor assignData(char data[][SIZE]);
14 void printCanvas(const Floor& canvas, const std::string& prompt);
15
16 int main(int argc, const char *argv[]) {
17     if (argc != 2) {
18         std::cout << "Need 'A'-'G' in command line parameter" << std::endl;
19         return -1;
20     }
21
22     //unit-testing for constructors and methods of Floor class
23     char type = *argv[1];
24     std::string prompt;
25
26     //IMPORTANT: If your output is inconsistent, temporarily replace srand(time(NULL))
27     //            with srand(0).
28     srand(time(NULL));
29
30     //'A' for default constructor and
31     //'B' for non-default constructor
32     if (type == 'A' || type == 'B') {
33         if (type == 'A') {
34             prompt = "Call default constructor Floor canvas;";
35             Floor canvas;
36             printCanvas(canvas, prompt);
37
38             //Sample output:
39             //Call default constructor Floor canvas;
40             //num of rows: 20
41             //num of cols: 20
42             //contents of canvas is
43             // , , , , , , , , , , , , , , , , , ,
44             // , , , , , , , , , , , , , , , , , ,
45             // , , , , , , , , , , , , , , , , , ,
46             // , , , , , , , , , , , , , , , , , ,

```

```

45 // , , , , , , , , , , , , , , , , , , , , ,
46 // , , , , , , , , , , , , , , , , , , , , ,
47 // , , , , , , , , , , , , , , , , , , , , ,
48 // , , , , , , , , , , , , , , , , , , , , ,
49 // , , , , , , , , , , , , , , , , , , , , ,
50 // , , , , , , , , , , , , , , , , , , , , ,
51 // , , , , , , , , , , , , , , , , , , , , ,
52 // , , , , , , , , , , , , , , , , , , , , ,
53 // , , , , , , , , , , , , , , , , , , , , ,
54 // , , , , , , , , , , , , , , , , , , , , ,
55 // , , , , , , , , , , , , , , , , , , , , ,
56 // , , , , , , , , , , , , , , , , , , , , ,
57 // , , , , , , , , , , , , , , , , , , , , ,
58 // , , , , , , , , , , , , , , , , , , , , ,
59 // , , , , , , , , , , , , , , , , , , , , ,
60 // , , , , , , , , , , , , , , , , , , , , ,
61
62     }
63     else if (type == 'B') {
64         std::cout << "Enter size of the canvas (needs to be at least 10, otherwise,
65             convert it to 20): ";
66         int size;
67         std::cin >> size;
68         prompt = "Call non-default constructor Floor canvas(" + std::to_string(size)
69             + ");";
70         Floor canvas(size);
71         printCanvas(canvas, prompt);
72
73         //Enter size of the canvas (needs to be at least 10, otherwise, convert it to 20): 11
74         //Call non-default constructor Floor canvas(11);
75         //num of rows: 11
76         //num of cols: 11
77         //contents of canvas is
78         // , , , , , , , , , , , , , , , , , , , , ,
79         // , , , , , , , , , , , , , , , , , , , , ,
80         // , , , , , , , , , , , , , , , , , , , , ,
81         // , , , , , , , , , , , , , , , , , , , , ,
82         // , , , , , , , , , , , , , , , , , , , , ,
83         // , , , , , , , , , , , , , , , , , , , , ,
84         // , , , , , , , , , , , , , , , , , , , , ,
85         // , , , , , , , , , , , , , , , , , , , , ,
86         // , , , , , , , , , , , , , , , , , , , , ,
87         //
88

```



```

89 //do not take the parameter as it is,
90 //need to make sure that parameter size is >= 10
91 //Sample input/output:
92 //Enter size of the canvas (needs to be at least 10, otherwise, convert it to 20): 5
93 //Call non-default constructor Floor canvas(5);
94 //num of rows: 20
95 //num of cols: 20
96 //contents of canvas is
97 // , , , , , , , , , , , , , , , , , , , , , ,
98 // , , , , , , , , , , , , , , , , , , , , , ,
99 // , , , , , , , , , , , , , , , , , , , , , ,
100 // , , , , , , , , , , , , , , , , , , , , , ,
101 // , , , , , , , , , , , , , , , , , , , , , ,
102 // , , , , , , , , , , , , , , , , , , , , , ,
103 // , , , , , , , , , , , , , , , , , , , , , ,
104 // , , , , , , , , , , , , , , , , , , , , , ,
105 // , , , , , , , , , , , , , , , , , , , , , ,
106 // , , , , , , , , , , , , , , , , , , , , , ,
107 // , , , , , , , , , , , , , , , , , , , , , ,
108 // , , , , , , , , , , , , , , , , , , , , , ,
109 // , , , , , , , , , , , , , , , , , , , , , ,
110 // , , , , , , , , , , , , , , , , , , , , , ,
111 // , , , , , , , , , , , , , , , , , , , , , ,
112 // , , , , , , , , , , , , , , , , , , , , , ,
113 // , , , , , , , , , , , , , , , , , , , , , ,
114 // , , , , , , , , , , , , , , , , , , , , , ,
115 // , , , , , , , , , , , , , , , , , , , , , ,
116 // , , , , , , , , , , , , , , , , , , , , , ,
117 //
118 }
119 }
120 else if (type >= 'C' && type <= 'E') {
121     char data[SIZE][SIZE];
122
123     //set some random values for data
124     char chosenValues[] = {'*', ' '};
125     int numChosenValues = sizeof(chosenValues) / sizeof(chosenValues[0]);
126     for (int row = 0; row < SIZE; row++) {
127         for (int col = 0; col < SIZE; col++) {
128             data[row][col] = chosenValues[rand() % numChosenValues];
129         }
130     }
131     Floor canvas = assignData(data);
132
133     if (type == 'C') {
134         //test clear method.

```

```

135     canvas.clear();
136
137     //after calling clearing method, each element of board should be ' '
138     bool isWrong = false;
139     for (int row = 0; row < canvas.getNumRows() && !isWrong; row++) {
140         for (int col = 0; col < canvas.getNumCols() && !isWrong; col++) {
141             if (canvas.at(row, col) != ' ') {
142                 std::cout << "clear method is not correct. Each element should be a
143                     space character\n";
144                 isWrong = true; //set a tag
145                 //break; //break only can break the inner loop, the outer loop still
146                     runs
147             }
148         }
149     }
150
151     if (!isWrong) {
152         std::cout << "clear method is correct\n";
153     }
154     else {
155         std::cout << "clear method is not correct\n";
156     }
157
158     //expected output:
159     //clear method is correct
160 }
161
162 else if (type == 'D') {
163     //test at method
164     bool isWrong = false;
165     for (int row = 0; row < canvas.getNumRows() && !isWrong; row++) {
166         for (int col = 0; col < canvas.getNumCols() && !isWrong; col++) {
167             if (canvas.at(row, col) != data[row][col]) {
168                 std::cout << "at method is not correct. The expected value is '" <<
169                     data[row][col] << "'\n";
170                 //set a tag
171                 isWrong = true;
172                 //break; //break only can break the inner loop, the outer loop
173                     still runs
174             }
175         }
176     }
177
178     if (!isWrong) {
179         std::cout << "at method is correct\n";
180     }
181     else {

```

```

176         std::cout << "at method is not correct\n";
177     }
178 }
179 else if (type == 'E') {
180     //canvas.print(); //debug purpose
181
182     // build expected output manually from data[][]
183     std::ostringstream expected;
184
185     // print actual via stream
186     std::string actual = canvas.to_string();
187
188     // build expected string from data[][]
189     expected << " ";
190     for (int col = 0; col < SIZE; col++) {
191         if (col % 10 == 0 && col / 10 >= 1) {
192             expected << col / 10;
193         }
194         else {
195             expected << ' ';
196         }
197     }
198     expected << '\n';
199
200     expected << " ";
201     for (int col = 0; col < SIZE; col++) {
202         expected << col % 10;
203     }
204     expected << '\n';
205
206     for (int row = 0; row < SIZE; row++) {
207         expected << (row % 10 == 0 && row / 10 >= 1 ? char('0' + row / 10) : ' ');
208         expected << row % 10;
209         for (int col = 0; col < SIZE; col++) {
210             expected << data[row][col];
211         }
212         expected << '\n';
213     }
214
215     //std::cout << "" << actual << ""\n\n";
216     //std::cout << "" << expected.str() << ""\n";
217     if (actual == expected.str()) {
218         std::cout << "to_string method is correct\n";
219     }
220     else {

```

```

221         std::cout << "to_string method is not correct\n";
222     }
223     //expected output:
224     //to_string method is correct
225 }
226
227 else if (type == 'F') {
228     //test mark method of Floor class
229     Floor canvas(SIZE);
230     canvas.mark(0, 0, '*');           // top-left corner
231     canvas.mark(SIZE-1, SIZE-1, '*'); // bottom-right corner, minimum size is
232         10
233     bool cornersCorrect = canvas.at(0,0) == '*' && canvas.at(SIZE-1, SIZE-1) == '*';
234
235     canvas.mark(-1, 0, '*'); // out of bounds, should do nothing
236     canvas.mark(0, SIZE, '*'); // out of bounds, should do nothing
237     canvas.mark(SIZE, 0, '*'); // out of bounds, should do nothing
238     canvas.mark(SIZE, SIZE, '*'); // out of bounds, should do nothing
239
240     // at() should return '\0' for invalid indices
241     bool outOfBoundsCorrect = canvas.at(-1,0) == '\0' &&
242         canvas.at(0,SIZE) == '\0' && canvas.at(SIZE,0) == '\0' && canvas.at(SIZE,SIZE)
243         == '\0';
244
245     int row = SIZE / 2;
246     int col = SIZE / 2;
247     canvas.mark(SIZE/2, SIZE/2, '*'); // out of bounds, should do nothing
248     bool centerCorrect = canvas.at(row, col) == '*';
249
250     row = rand() % SIZE;
251     col = rand() % SIZE;
252     canvas.mark(row, col, '*'); // ok even if (row,col) overlaps (0,0) or (SIZE-1,
253         SIZE-1) since they are already '*'
254
255     bool randomCorrect = canvas.at(row, col) == '*';
256     if (cornersCorrect && outOfBoundsCorrect && centerCorrect && randomCorrect) {
257         std::cout << "mark method is correct\n";
258     }
259     else {
260         std::cout << "mark method is not correct\n";
261     }
262
263     //sample output:
264     //mark method is correct
265 }

```

```

264     else {
265         std::cout << "Unrecognized option: " << type << std::endl;
266         return -1;
267     }
268
269     return 0;
270 }
271
272 Floor assignData(char data[][SIZE]) {
273     Floor canvas(SIZE);
274
275     for (int row = 0; row < canvas.getNumRows(); row++) {
276         for (int col = 0; col < canvas.getNumCols(); col++) {
277             canvas.mark(row, col, data[row][col]); //set board[row][col] of *canvasPtr --
                a Floor object -- to be data[row][col]
278         }
279     }
280
281     return canvas;
282 }
283
284 void printCanvas(const Floor& canvas, const std::string& prompt) {
285     std::cout << prompt << '\n';
286     std::cout << "num of rows: " << canvas.getNumRows() << '\n';
287     std::cout << "num of cols: " << canvas.getNumCols() << '\n';
288     std::cout << "contents of canvas is\n";
289     for (int row = 0; row < canvas.getNumRows(); row++) {
290         for (int col = 0; col < canvas.getNumCols(); col++) {
291             std::cout << canvas.at(row, col) << ',';
292         }
293         std::cout << '\n';
294     }
295 }

```

Run the following commands.

```
1 g++ -o floor Floor.cpp FloorUnitTest.cpp
```

If the code compiles, you would get a `floor` executable file. To test the default constructor, run

```
1 ./floor A
```

Test the rest of constructors and methods according to the comments in `FloorUnitTest.cpp`.

4.4 Submission for Task A

Submit `Floor.cpp` to gradescope.

5 Task B: Implement Turtle Class

5.1 Download Turtle.hpp

Download link: [Turtle.hpp](#) Its contents are as follows.

```
1 // File name: Turtle.hpp
2 #ifndef Turtle_HPP
3 #define Turtle_HPP
4
5 #include <ostream> //std::ostream, used in << operator for Direction
6
7 /**
8  * @brief Represents the state of the turtle's drawing pen.
9  *
10 * UP - the pen is lifted; movement does not draw.
11 * DOWN - the pen is lowered; movement draws a trail.
12 */
13 enum class Pen { UP, DOWN };
14
15 /**
16 * @brief Represents the four cardinal directions the turtle can face.
17 */
18 enum class Direction { NORTH, EAST, SOUTH, WEST };
19
20 /**
21 * @brief Overloads the stream insertion operator for Direction.
22 * @param os The output stream to write to.
23 * @param heading The Direction value to print.
24 * @return Reference to the output stream, with the direction written
25 * as a human-readable string: "NORTH", "EAST", "SOUTH", or "WEST".
26 */
27 std::ostream& operator<<(std::ostream& os, Direction heading);
28
29 /**
30 * @brief Models a turtle that moves on a 2-D grid and optionally draws a trail.
31 *
32 * The turtle tracks its position (row, col), the direction it faces (heading),
33 * and whether its pen is up or down. All data members are initialized at the
34 * point of declaration so no user-defined constructor is required.
35 */
36 class Turtle {
37 public:
38
39 // No user-defined constructor is needed.
40 // All data members are initialized where they are declared (modern C++ style),
41 // and the compiler-generated default constructor is sufficient.
```

```

42
43 /**
44  * @brief Resets the turtle to its initial state.
45  *
46  * After this call:
47  *   - pen      == Pen::UP
48  *   - heading == Direction::EAST
49  *   - row      == 0
50  *   - col      == 0
51  */
52 void reset();
53
54 /**
55  * @brief Reports whether the pen is currently lifted.
56  * @return true if pen == Pen::UP (turtle does not draw while moving).
57  * @return false if pen == Pen::DOWN (turtle draws while moving).
58  */
59 bool isPenUp() const;
60
61 /**
62  * @brief Lifts the pen so that subsequent moves do not draw.
63  * @post pen == Pen::UP
64  */
65 void liftUpPen();
66
67 /**
68  * @brief Lowers the pen so that subsequent moves draw a trail.
69  * @post pen == Pen::DOWN
70  */
71 void putDownPen();
72
73 /**
74  * @brief Moves the turtle forward by the given number of steps in its current
       heading.
75  *
76  * The axis updated depends on the current heading:
77  *   - EAST : col += steps
78  *   - WEST : col -= steps
79  *   - SOUTH : row += steps
80  *   - NORTH : row -= steps
81  *
82  * Negative values for steps move the turtle in the opposite direction.
83  * No bounds checking is performed; the caller is responsible for
84  * keeping the turtle within any grid boundaries.
85  *
86  * @param steps Number of grid cells to move (may be negative).

```

```

87     */
88 void move(int steps);
89
90 /**
91  * @brief Rotates the turtle 90 degrees to the left (counter-clockwise).
92  *
93  * Heading transitions:
94  *   EAST -> NORTH
95  *   NORTH -> WEST
96  *   WEST -> SOUTH
97  *   SOUTH -> EAST
98  *
99  * @post heading is updated to the next counter-clockwise direction.
100 */
101 void turnLeft();
102
103 /**
104  * @brief Rotates the turtle 90 degrees to the right (clockwise).
105  *
106  * Heading transitions:
107  *   EAST -> SOUTH
108  *   SOUTH -> WEST
109  *   WEST -> NORTH
110  *   NORTH -> EAST
111  *
112  * @post heading is updated to the next clockwise direction.
113  */
114 void turnRight();
115
116 /**
117  * @brief Sets the turtle's row position on the grid.
118  *
119  * No bounds checking is performed; the caller is responsible for
120  * passing a valid row index.
121  *
122  * @param row The new row index (0-based).
123  */
124 void setRow(int row);
125
126 /**
127  * @brief Sets the turtle's column position on the grid.
128  *
129  * No bounds checking is performed; the caller is responsible for
130  * passing a valid column index.
131  *
132  * @param col The new column index (0-based).

```



```

133     */
134 void setCol(int col);
135
136 /**
137  * @brief Returns the turtle's current row position.
138  * @return The current row index (0-based).
139  */
140 int getRow() const;
141
142 /**
143  * @brief Returns the turtle's current column position.
144  * @return The current column index (0-based).
145  */
146 int getCol() const;
147
148 /**
149  * @brief Sets the direction the turtle is facing.
150  *
151  * Because Direction is a scoped enum, only the four defined values
152  * (NORTH, EAST, SOUTH, WEST) are valid; no explicit validation is needed.
153  *
154  * @param heading The new heading for the turtle.
155  */
156 void setHeading(Direction heading);
157
158 /**
159  * @brief Returns the direction the turtle is currently facing.
160  * @return The current heading as a Direction value.
161  */
162 Direction getHeading() const;
163
164 private:
165     // Modern C++ preferred style: initialize members where they are declared,
166     // and let the compiler generate the default constructor.
167     Pen      pen      = Pen::UP;          ///< Current pen state (UP or DOWN).
168     Direction heading = Direction::EAST;  ///< Current heading (NORTH/EAST/SOUTH/WEST).
169     int      row      = 0;                ///< Current row position on the grid (0-based).
170     int      col      = 0;                ///< Current column position on the grid (0-based)
171     ).
172 };
173 #endif

```

5.2 Implement Turtle.cpp

Read the requirements in `Turtle.hpp`. Implement all methods declared in `Turtle.hpp` inside `Turtle.cpp`.
For your convenience, here are the code to overload `>>` operator for enum `Direction` class.

```
1 std::ostream& operator<<(std::ostream& os, Direction heading) {
2     switch (heading) {
3         case Direction::EAST: return os << "EAST";
4         case Direction::WEST: return os << "WEST";
5         case Direction::SOUTH: return os << "SOUTH";
6         case Direction::NORTH: return os << "NORTH";
7         default: return os << "Unknown";
8     }
9 }
```

5.3 Test Turtle.cpp using TurtleUnitTest.cpp

Download link: [TurtleUnitTest.cpp](#) Its contents are as follows.

```
1 //file name: TurtleUnitTest.cpp
2 #include <iostream>
3 #include <cstdlib> //srand
4 #include <ctime> //time
5 #include "Turtle.hpp"
6 //g++ -o turtle Turtle.cpp TurtleUnitTest.cpp
7 //test default constructor using
8 //./turtle A
9
10 int main(int argc, const char *argv[]) {
11     if (argc != 2) {
12         std::cout << "Need 'A'-'G' in command line parameter" << std::endl;
13         return -1;
14     }
15
16     //unit-testing for methods of Turtle class, which has no constructors
17     char type = *argv[1];
18     std::string prompt;
19
20     srand(time(NULL));
21     const int SIZE = 20;
22     switch (type) {
23         case 'A': { //test reset method
24             Turtle tur;
25             tur.reset();
26             if (tur.isPenUp() && tur.getRow() == 0 && tur.getCol() == 0 && tur.getHeading() ==
                Direction::EAST) {
27                 std::cout << "reset method is correct" << std::endl;

```

```

28     }
29     else {
30         std::cout << "reset method is not correct" << std::endl;
31     }
32     break;
33 }
34 //When running ./turtle A,
35 //where turtle obtained by
36 //g++ -o turtle TurtleUnitTest.cpp Turtle.cpp
37 //expected output is as follows
38 //reset method is correct
39
40
41 case 'B': { //test isPenUp, liftUpPen, and putDownPen methods
42     Turtle tur;
43     tur.liftUpPen();
44     bool penUpTrue = tur.isPenUp();
45
46     tur.putDownPen();
47     bool penUpFalse = tur.isPenUp();
48
49     if (penUpTrue && penUpFalse == false) {
50         std::cout << "isPenUp, liftUpPen, and putDownPen methods are correct" << std::
            endl;
51     }
52     else {
53         std::cout << "isPenUp, liftUpPen, and putDownPen methods are not correct" << std
            ::endl;
54     }
55
56     break;
57 }
58 //Running the following two comands
59 //g++ -o turble TurtleUnitTest.cpp Turtle.cpp
60 //./turtle B
61 //expected output: isPenUp, liftUpPen, and putDownPen methods are correct
62
63 case 'C': { //test setRow and getRow methods
64     Turtle tur;
65     int row = rand() % SIZE;
66     tur.setRow(row);
67     if (tur.getRow() == row) {
68         std::cout << "setRow and getRow methods are correct" << std::endl;
69     }
70     else {
71         std::cout << "setRow and getRow methods are not correct" << std::endl;

```

```

72     }
73
74     break;
75 }
76 //expected output after running ./turtle C
77 //setRow and getRow methods are correct
78
79 case 'D': { //test setCol and getCol methods
80     Turtle tur;
81     int col = rand() % SIZE;
82     tur.setCol(col);
83     if (tur.getCol() == col) {
84         std::cout << "setCol and getCol methods are correct" << std::endl;
85     }
86     else {
87         std::cout << "setCol and getCol methods are not correct" << std::endl;
88     }
89
90     break;
91 }
92 //expected output after running ./turtle D
93 //setCol and getCol methods are correct
94
95 case 'E': { //test turnLeft, setHeading and getHeading methods
96     Turtle tur;
97
98     tur.setHeading(Direction::EAST);
99     tur.turnLeft();
100     if (tur.getHeading() != Direction::NORTH) {
101         std::cout << "When direction is EAST, turnLeft method is not correct." << std::
102             endl;
103         break;
104     }
105
106     tur.setHeading(Direction::WEST);
107     tur.turnLeft();
108     if (tur.getHeading() != Direction::SOUTH) {
109         std::cout << "When direction is WEST, turnLeft method is not correct." << std::
110             endl;
111         break;
112     }
113
114     tur.setHeading(Direction::NORTH);
115     tur.turnLeft();
116     if (tur.getHeading() != Direction::WEST) {
117         std::cout << "When direction is NORTH, turnLeft method is not correct." << std::

```

```

        endl;
116     break;
117 }
118
119 tur.setHeading(Direction::SOUTH);
120 tur.turnLeft();
121 if (tur.getHeading() != Direction::EAST) {
122     std::cout << "When direction is SOUTH, turnLeft method is not correct." << std::
        endl;
123     break;
124 }
125
126 std::cout << "turnLeft method is correct." << std::endl;
127 break;
128 }
129 //expected output after running ./turtle E:
130 //turnLeft method is correct.
131
132 case 'F': { //test turnRight, setHeading and getHeading methods
133     Turtle tur;
134
135     tur.setHeading(Direction::EAST);
136     tur.turnRight();
137     if (tur.getHeading() != Direction::SOUTH) {
138         std::cout << "When direction is EAST, turnRight method is not correct." << std::
            endl;
139         break;
140     }
141
142     tur.setHeading(Direction::WEST);
143     tur.turnRight();
144     if (tur.getHeading() != Direction::NORTH) {
145         std::cout << "When direction is WEST, turnRight method is not correct." << std::
            endl;
146         break;
147     }
148
149     tur.setHeading(Direction::NORTH);
150     tur.turnRight();
151     if (tur.getHeading() != Direction::EAST) {
152         std::cout << "When direction is NORTH, turnRight method is not correct." << std
            ::endl;
153         break;
154     }
155
156     tur.setHeading(Direction::SOUTH);

```

```

157     tur.turnRight();
158     if (tur.getHeading() != Direction::WEST) {
159         std::cout << "When direction is SOUTH, turnRight method is not correct." << std
160             ::endl;
161         break;
162     }
163
164     std::cout << "turnRight method is correct." << std::endl;
165     break;
166 }
167 //expected output after running ./turtle F:
168 //turnRight method is correct.
169
170 case 'G': { // void move(int steps);
171     Turtle tur;
172
173     //move in east direction
174     tur.setHeading(Direction::EAST);
175     int numStepsEast = 0;
176     while (numStepsEast == 0) {
177         numStepsEast = rand() % SIZE;
178     }
179     //Make sure numStepsEast is not zero.
180
181     tur.move(numStepsEast);
182
183     if (tur.getRow() != 0 || tur.getCol() != numStepsEast) {
184         std::cout << "When direction is Direction::EAST, move method of " <<
185             numStepsEast << " steps is wrong." << std::endl;
186         break;
187     }
188
189     //move in south direction from (0, numStepsEast)
190     tur.setHeading(Direction::SOUTH);
191     int numStepsSouth = 0;
192     while (numStepsSouth == 0) {
193         numStepsSouth = rand() % SIZE;
194     }
195     //Make sure numStepsSouth is not zero.
196
197     tur.move(numStepsSouth);
198     if (tur.getRow() != numStepsSouth || tur.getCol() != numStepsEast) {
199         std::cout << "When direction is Direction::SOUTH, move method of " <<
200             numStepsSouth << " steps is wrong." << std::endl;
201         break;
202     }
203 }

```

```

200 //set tur to the rightmost position of the first row, where row index is 0 and
201     column index is SIZE-1.
202 tur.setRow(0);
203 tur.setCol(SIZE-1);
204
205 //move in west direction
206 tur.setHeading(Direction::WEST);
207
208 int numStepsWest = 0;
209 while (numStepsWest == 0) {
210     numStepsWest = rand() % SIZE;
211 }
212 //Make sure numStepsWest is not zero.
213
214 tur.move(numStepsWest);
215 if (tur.getRow() != 0 || tur.getCol() != SIZE -1 -numStepsWest) {
216     std::cout << "When direction is Direction::WEST, move method of " <<
        numStepsWest << " steps is wrong." << std::endl;
217     break;
218 }
219
220 //set tur to the bottom right position in the grid, where row index is SIZE-1
221     and column index is SIZE-1.
222 tur.setRow(SIZE-1);
223 tur.setCol(SIZE-1);
224
225 //move in north direction
226 tur.setHeading(Direction::NORTH);
227 int numStepsNorth = 0;
228 while (numStepsNorth == 0) {
229     numStepsNorth = rand() % SIZE;
230 }
231 //Make sure numStepsNorth is not zero.
232
233 tur.move(numStepsNorth);
234 if (tur.getRow() != SIZE -1 -numStepsNorth || tur.getCol() != SIZE -1) {
235     std::cout << "When direction is Direction::NORTH, move method of " <<
        numStepsNorth << " steps is wrong." << std::endl;
236     break;
237 }
238
239 std::cout << "move method is correct" << std::endl;
240
241 break;
242 }

```

```

242 //expected output after running ./turtle G:
243 //move method is correct
244
245     default:
246         std::cout << "Unrecognized option: " << type << std::endl;
247     }
248
249     return 0;
250 }

```

Compile and link `Turtle.cpp` and `TurtleUnitTest.cpp` using the following command.

```
g++ -o turtle Turtle.cpp TurtleUnitTest.cpp
```

Run each test case by passing the corresponding letter as a command-line argument. For example, to test the `reset` method, run

```
./turtle A
```

Test cases A through G correspond to the following methods:

- A — `reset`
- B — `isPenUp`, `liftUpPen`, `putDownPen`
- C — `setRow`, `getRow`
- D — `setCol`, `getCol`
- E — `turnLeft`, `setHeading`, `getHeading`
- F — `turnRight`, `setHeading`, `getHeading`
- G — `move`

5.4 Submission for Task B

Submit `Turtle.cpp` to gradescope.

6 Task C: Implement TurtleGraphics Class

6.1 Download TurtleGraphics.hpp

Download link: [TurtleGraphics.hpp](#)

6.2 Implement TurtleGraphics.cpp

Read the requirements in `TurtleGraphics.hpp`. Implement all methods declared in `TurtleGraphics.hpp` inside `TurtleGraphics.cpp`.

Hint: The private method `processCommand` is called internally by both `run()` and `run(commands)`. Notice that it has a parameter `inputFromConsole`:

- When called from `run()`, command 5 reads the number of steps from `std::cin`.
- When called from `run(commands)`, command 5 uses the `numSteps` parameter directly.

Think about what value of `inputFromConsole` corresponds to each case, and how `numSteps` should be set in each call.

6.3 Download input3.txt

These are the commands to draw digit 3 in a 20 x 20 grid.

Download link: [input3.txt](#)

6.4 Test TurtleGraphics.cpp

Download link: [TurtleGraphicsUnitTest.cpp](#)

Compile and link all source files using the following command.

```
g++ -o tg Floor.cpp Turtle.cpp TurtleGraphics.cpp TurtleGraphicsUnitTest.cpp
```

Run the resulting executable to verify your implementation.

Test default constructor of `TurtleGraphics` class using

```
./tg A
```

Test restart method using

```
./tg B
```

Test run commands stored in a two-dimensional vector to draw digit 5 using

```
./tg C
```

Test run commands with console input to draw digit 3 using

```
./tg D < input3.txt
```

Or you can test the commands by inputting one a time from the console using

```
./tg D
```

6.5 Submission for Task C

Submit `TurtleGraphics.cpp` to gradescope.